State, Not Syntax: Harness Optimization for Small-Model Multi-Turn Tool Calling

Sanskar Thapa

Master's thesis draft (December 2026)

`https://github.com/sskarz/Thesis`

December 2026

Abstract

Assistants that *act* emit *tool calls*: a function name plus arguments. A *harness*, the program around the model, runs each call. On a laptop this keeps data local, but a small model often fails when later calls depend on earlier ones. We study Gemma-4-E2B-it (about two billion effective parameters) on 200 multi-turn tasks from the Berkeley Function Calling Leaderboard (BFCL v4). Grading uses the final condition of a simulated backend, its *state*, such as whether a car's engine is on. Single-call tests score 772 of 840 (91.9%). Through the server's built-in tool-call field the model finishes 54 of 200 (27.0%), and forcing valid syntax lowers accuracy further: failures concern state, not syntax. The harness, with the weights fixed, uses JSON-text calls, writes a state summary into the model's context, stops repeated calls, and shows a four-line card advising whether to call, ask, finish or refuse. Together these reach 77 of 200 (38.5%). The 8.0 percentage-point gain over JSON text is suggestive ($p = 0.023$ uncorrected; not significant after correcting for multiple comparisons). It comes mostly from calling or stopping at the right moment; wrong final states remain the largest failure type. Fine-tuning did not reach that level. A control fine-tune that should change nothing if the machinery is sound stays close to the untuned model, so the rebuilt mix's data are the leading suspect but not the only one: that mix also differed in training length.

1 Introduction

To check a calendar or lock a car, a language model emits a *tool call*: a function name plus arguments. A surrounding program, the *harness*, runs the call against a *backend* (here a simulated vehicle or filesystem) and feeds the result back to the model (Figure 1). Real tasks chain several dependent calls, which we call *multi-turn* tool calling: look up a value, use it, check the outcome, and know when to stop.

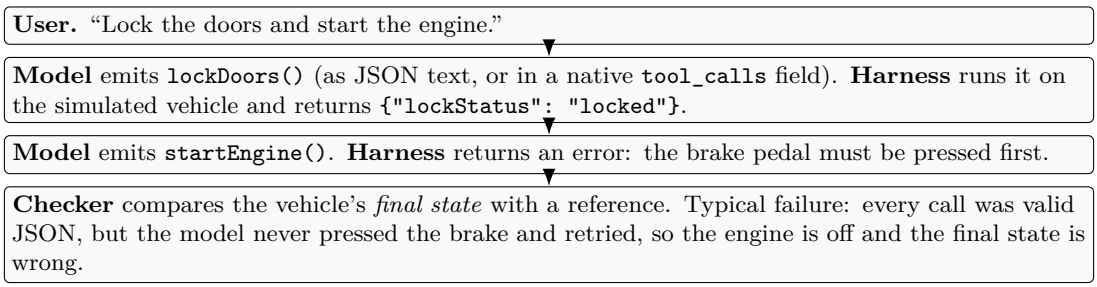


Figure 1: The multi-turn tool-calling loop studied in this paper. Grading uses the final state of the simulated backend; whether the calls parse is not enough. Illustrative, not a scored task.

Running this loop locally with a small model keeps private data on the device, works offline and has no per-request cost. The catch is accuracy. We study Gemma-4-E2B-it [1] ("E2B": about two billion effective parameters, meaning the inference footprint; "it": instruction-tuned) on BFCL v4 [2], served with the llama.cpp inference engine [3] on a MacBook Pro. Through the server's built-in tool-call field, the native tool interface, it completes 27.0% of 200 multi-turn tasks.

Is the model failing to *write* calls or to *choose* them? Section 4 shows it is mostly the latter: tasks fail because a needed tool never ran or the backend ended in the wrong state. We therefore treat the problem as *harness optimization*. The weights stay fixed; we change the loop around the model: how calls are parsed, what state the harness records and shows back, when it ends a turn, and how the next step is framed. Two of these levers, a state summary and a status card, reach the model as text in its context, so the harness acts partly through the context; both are computed from executed results. We compare harness control with two other families of fixes: forcing syntactically valid output, and fine-tuning the weights. Our contributions:

1. **A failure anatomy within one model: state, not syntax.** With the same model, quantization and server, single-call accuracy is 91.9% and multi-turn accuracy is 27.0%. On the multi-turn tasks the native failures are mostly empty turns (59) and wrong final states (68); under-calls are 19. Two ways of forcing each call to be valid syntax lower accuracy to 10.5% and 13.5% and shift failures into under-calls (Section 4).

2. **A training-free harness ladder with paired statistics.** With greedy (temperature 0) decoding for every harness configuration, JSON-text calls reach 30.5%, a three-part minimal recipe 35.5% and the full stack 38.5% (77/200) on its first run. Writing the same tool list more compactly on that stack added about 7 points in two paired runs (84 and 88 of 200, against 71 and 72 for reruns of the original list): the gain replicated (Section 5). The full stack's gain of 8.0 percentage points (pp) over JSON text is suggestive ($p = 0.023$ uncorrected; not significant after correcting for multiple comparisons), and it matches a five-sample majority vote (36.5%) at about 1/7 of the generation time (Section 5).

3. **A controlled negative result on fine-tuning a 2B tool caller.** The best supervised fine-tune scores 15.0%, significantly below JSON text even after correction; preference tuning scores 26.5%. Models trained on a larger rebuilt data mix fail a 20-task screening check (1–2 of 20, against 17 for the untuned model), while a control fine-tune that should change nothing if the machinery is sound stays close to the untuned model (15 of 20). The rebuilt mix's data are the leading suspect but not the only one, because that mix also differed in training length (Section 6).

2 Background and Related Work

Benchmark. The Berkeley Function Calling Leaderboard (BFCL) evaluates language models as API callers [2, 4]. Its single-turn categories check one emitted call against a reference with an abstract-syntax-tree (AST) matcher. Its multi-turn categories run the model in an agent loop against simulated backends (for example VehicleControl and GorillaFileSystem, a sandboxed Unix-like filesystem) and compare the backends' final state with a reference. A *turn* is one user request and everything the model does before handing control back; each multi-turn task has several turns. We use the *multi-turn base* split (200 tasks, all tools available). Accuracy is the fraction of the 200 tasks whose final state matches.

Output channel. With *native function calling* (NFC), the server receives a tool list and the model returns structured `tool_calls`, as in most production APIs. With *instruction-followed function calling* (IFFC), the prompt asks the model to write its calls as a JSON list in ordinary text, which the harness parses. Taheri et al. report that small models follow such JSON instructions more reliably than native tool APIs on single-turn BFCL [5]; we use their single-model variant.

Constrained decoding. Grammar-constrained generation restricts each next token so that output is always valid, for example to a JSON schema or a list of tool names [6, 7]; Yu et al. constrain tool names with a token-level automaton [8]. Format restrictions can also reduce answer quality [9]. Section 4 tests whether forcing validity helps in a multi-turn loop.

Harness-side control and test-time compute. Context-compression methods summarize long histories [10]; our state summary is instead written by the harness from executed results. Observe–orient–decide–act (OODA) controllers structure agent steps [11]; our status card borrows only the name. Guided templates add a reasoning scaffold before calls [12]. Sampling several answers and voting is a standard way to buy accuracy with compute [13]; our vote of five is that baseline.

Training for tool use. Specialist small callers such as xLAM [14] and Hammer [15] are trained on synthetic tool data such as ToolACE [16] and APIGen [17]. *Supervised fine-tuning* (SFT) trains the model to reproduce target responses; we use low-rank adapters (LoRA) [18], small trainable matrices added to frozen weights. *Direct Preference Optimization* (DPO) trains on pairs of preferred and rejected responses [19, 20]. Terms used in this paper are collected in a glossary (Appendix C).

3 Setup and Methods

Model and serving. All results use Gemma-4-E2B-it, quantized with Q4_K_M (a 4-bit weight format of llama.cpp), served by `llama-server` with Metal acceleration, Apple's GPU interface, on a MacBook Pro (Apple M4 Pro, 48 GB). Earlier runs in this paper are recorded as llama.cpp build 8111 (commit `11c325c6e`). Runs 78 and 80–85 used commit `d344123fe` (flags in the reproducibility note). Decoding is greedy (temperature 0) except for majority voting. Temperature 0 does not make repeat runs identical: with four parallel server slots, the same full stack scored 77/200, then 71/200 and 72/200. Most runs used four parallel server slots (requests the server processes concurrently) and 16,384 tokens of context per request (65,536 shared across four slots); the native-API baseline used eight slots, a serving difference that could affect its results. This paper did not test whether the eight-slot and four-slot greedy outputs match.

Harness interventions. All leave the weights untouched. A task is correct only when the BFCL multi-turn checker returns valid. The script `evaluate_multi_turn.py` calls that checker on the model's calls and the task's ground truth. Appendix A shows the text each intervention except the shorter tool list writes, with examples. By design, the state summary targets lost state, the stop rule targets runaway repetition, and the status card targets the choice between acting and abstaining.

- **JSON text (IFFC)** instead of the native tool API (NFC); the harness parses the calls.
- **State summary.** After each executed call, the harness writes a short `[STATE]` snapshot of facts from real results (current directory, known files, last calls) into the conversation.

- **Stop on repeats.** The harness ends the current turn when the model repeats its previous set of calls, or when it re-proposes calls already executed in this turn after at least six tool steps. An *ungated* variant drops the six-step minimum.
- **Status card.** A four-line card (goal, known facts, last call and outcome, and an advisory next move: call, ask, final or refuse) is prefixed to the latest user message. The harness fills it from fixed rules on the conversation and executed results; it never sees reference answers. JSON text + stop on repeats + card is our *minimal recipe*.
- **Full stack.** JSON text + state summary + stop on repeats + status card, greedy. On its first run this is the harness we compare with JSON text.
- **Shorter tool list.** On the full stack, every tool stays in the prompt. The harness states each tool group's shared description once, instead of repeating it, and prints each schema without indentation. The model, the four pieces above, and the tasks stay the same. The shorter list is text in the context, so this lever, like the state summary and the status card, is harness-side control delivered through the context.
- **Vote of five.** On top of the state summary, at each step the harness samples five candidate calls at temperature 0.7 and executes the most common one. This is the compute-heavy comparison point.
- **Forced-valid decoding.** *Constrained decoding* restricts output in two passes (tool names, then arguments by schema); a *tool-name automaton* constrains only the name, inspired by [8].

Training. We train LoRA adapters on Gemma-4-E2B-it with MLX [21], Apple's machine-learning framework, merge them into the weights, quantize to Q4_K_M and evaluate with JSON text. Early mixes combine ToolACE dialogues with sampled xLAM data; a later *rebuilt mix* (30,000 rows) combines about 45% newly simulated, execution-checked episodes with cleaned public data. We also train one DPO model. Because full 200-task runs are slow, trained models first face a fixed 20-task *gate* drawn from the same split and run with the status card; the bar is 17/20, the untuned model's score with the card. A smaller 10-task regression check is described in Appendix B.1.

Error labels. Each failed task gets one primary label from the checker's message: *empty turn* (no call on a turn that needed one), *under-call* (a required tool never ran), or *state mismatch* (tools ran but the backend's final state differs from the reference). Output the harness cannot parse counts as no call, so a parse failure can only appear as an empty turn.

Statistics. We report Wilson 95% intervals [22], which stay accurate for small counts. For paired comparisons on the same 200 tasks we use exact two-sided McNemar tests [23], which use only the tasks on which two conditions disagree. Eight comparisons form the family. Four involve the full stack, so the family was fixed after that configuration was built. The family covers the two headline configurations against their baselines, the output-channel switch and the best fine-tune; the four single-lever comparisons in Table 4 (state summary against JSON text, the vote against the state summary, stop on repeats against JSON text, and the minimal recipe against stop on repeats) are exploratory and were left out. We report uncorrected p-values and note where correcting for the eight comparisons (Holm's method [24], which keeps the chance of any false positive among them at 5%) changes the conclusion. The shorter-list test uses the 0.05 level set in the plan and is not one of these eight tests. The plan paired it with the first full-stack run; after the smoke check failed its gate, the comparison run was changed to a fresh rerun of the original list. A second pair repeats

the test in the opposite order (shorter list first). To combine the two pairs, we count for each task how many of its two runs each list solved and apply an exact two-sided sign test to the tasks where the counts differ.

4 Where Small Models Fail

One call is easy; a chain is not. On three single-call BFCL categories (840 cases), Gemma-4-E2B-it through the native API scores 772/840 (91.9%, interval 89.9–93.6%; Table 5). *Simple* offers one function and asks for that call (375/400). *Multiple* offers several functions and asks for the right one (183/200). *Irrelevance* offers functions when the query needs no call (214/240). The two parallel-call categories are excluded because earlier runs on this server hit a tool-grammar bug that truncated multi-call output. On multi-turn tasks, the *same model, quantization and server* completes only 54 of 200 (27.0%). The two settings differ in grading as well as length, but the gap is consistent with carrying a chain of calls through a changing state, rather than choosing one call, being the bottleneck.

Most failures are about the world. Figure 2 breaks down the failed tasks (full counts in Table 3). For every JSON-text condition, state mismatch is the largest bucket (53–109 tasks). With plain JSON text it is followed by empty turns (41) and under-calls (38); the native API has more empty turns (59) and far fewer under-calls (19). Harness control mainly removes empty turns: the full stack has 20, against 41 for JSON text, while its state mismatches are 68 (60 for JSON text). Because parse failures can only appear as empty turns, most of what remains is not a formatting problem.

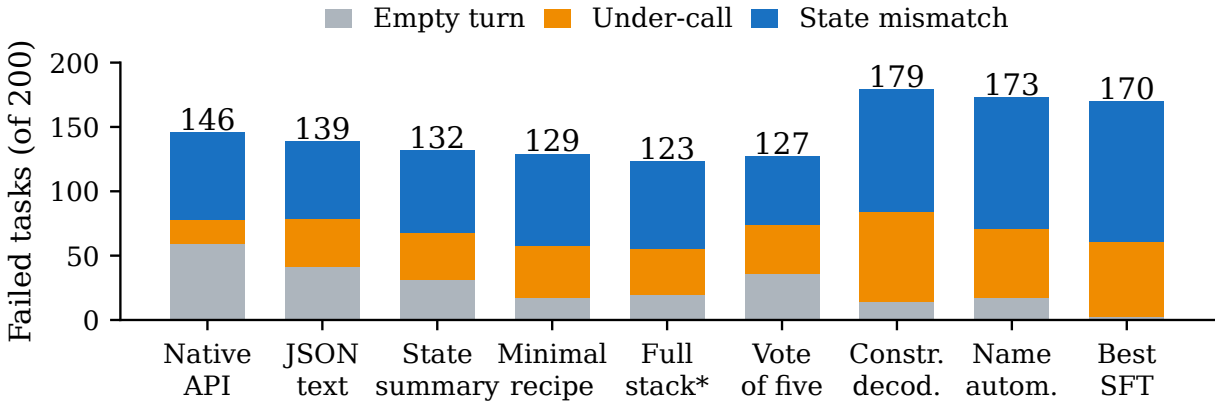


Figure 2: Why tasks fail. Each bar is one condition on the same 200 tasks (* marks the first full-stack run; Constr. decod. = constrained decoding; Name autom. = tool-name automaton; Best SFT = best fine-tune; conditions as in Section 3); its height is the number of failed tasks, split by primary error label (Section 3). Harness control mostly removes empty turns (grey); wrong final states (blue) remain. Forced-valid output and the best fine-tune shift failures into under-calls (orange) and state mismatches. The shorter tool list is omitted. Checker rows exist for its runs (116 and 112 failed tasks) and the original-list reruns (129 and 128), but those rows were not labelled for this revision.

Forcing valid syntax hurts. If failures were mostly malformed output, guaranteeing valid output should help. It does the opposite. Two-pass constrained decoding scores 21/200 (10.5%; interval

7.0–15.5%), 20.0 pp below JSON text, with 70 under-calls and 95 state mismatches. The tool-name automaton scores 27/200 (13.5%; 9.4–18.9%), with state mismatch rising to 102. Our two-pass program, which forces each call into valid syntax, approximates rather than reproduces full grammar engines [7], but the effect is large and in the wrong direction: a valid call is not a correct next step.

5 What Helps: Harness-Side Control

If the bottleneck is choosing the next step, the harness, which sees every executed result, is a natural place to intervene. Figure 3 shows the results. Every change is greedy except the vote of five, and none touches the weights; exact counts and intervals for every condition are in Table 5 (Appendix C).

JSON text over the native API (+3.5 pp). Writing calls as JSON text scores 61/200 against 54/200 natively ($p = 0.32$, not significant). Part of the gap may come from serving, and a guided template on the native API ties JSON text (Section 7), so we do not claim the channel switch alone as a gain. We build the harness ladder on JSON text because the harness can parse and inspect it directly.

State summary (+3.5 pp). The harness-written snapshot scores 68/200 ($p = 0.30$ against JSON text). Empty turns fall from 41 to 31 while state mismatches do not (60 to 64), suggesting that the summary mainly keeps the model calling.

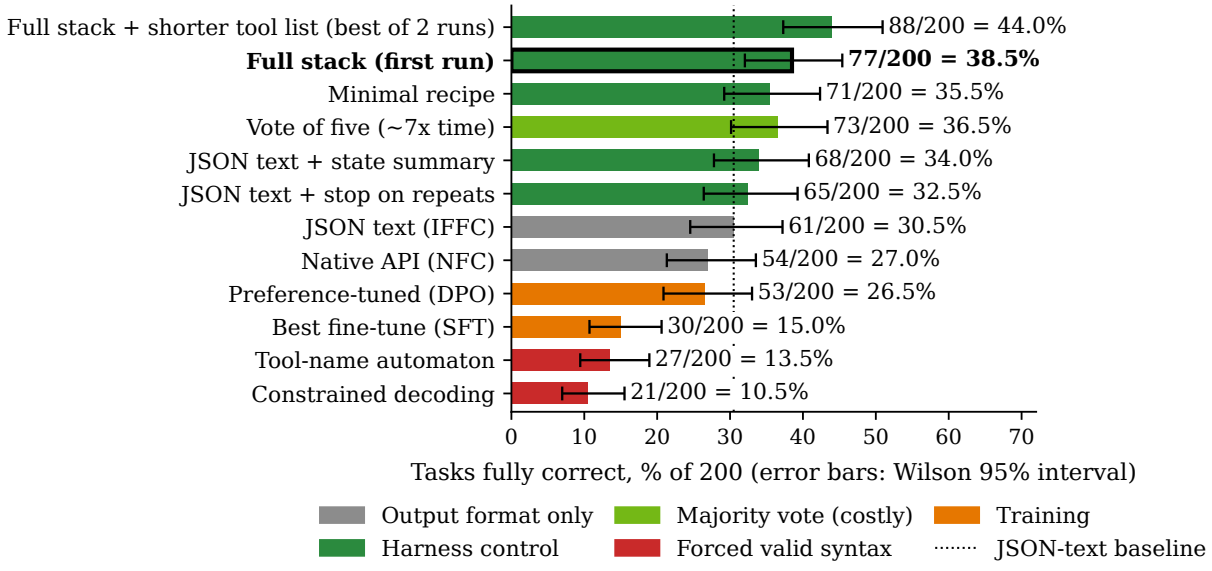


Figure 3: Multi-turn accuracy on 200 BFCL v4 tasks, Gemma-4-E2B-it. Each bar is a separate run, not a stack of the bars below it. Bars: share of tasks fully correct; error bars: Wilson 95% intervals. Dark green: harness control on JSON text (the first full-stack run is outlined and in bold); light green: majority vote, the compute-heavy comparison; grey: output format alone; red: forced-valid output; orange: training. Dotted line: JSON-text baseline (30.5%). Top bar: the full stack with a shorter tool list, the better of its two runs (88/200; the other scored 84/200, and two reruns of the original list scored 71/200 and 72/200; Table 1).

Stop on repeats and the status card (minimal recipe, 35.5%). The gated stop rule scores 65/200 (+2.0 pp, $p = 0.48$). Stopping at the *first* repeat instead scores 54/200, 11 tasks below the gated rule, and all 37 forced stops fell on tasks that failed. Adding the status card gives the minimal recipe, 71/200 (+5.0 pp over JSON text, $p = 0.16$). Relative to stop on repeats alone, empty turns fall from 38 to 17 while state mismatches rise from 60 to 71, so the error labels suggest the card acts mainly as an act-versus-abstain control.

Full stack (38.5% on the first run). Combining all four greedy changes scores 77/200 (38.5%, interval 32.0–45.4%). Paired against JSON text, it solves 30 tasks that JSON text misses and misses 14 that JSON text solves ($p = 0.023$ uncorrected; not significant after correction). Against the minimal recipe, the state summary alone and the vote of five, the differences are within noise (Table 2). The configuration was assembled after we had seen earlier results on these tasks, so its lead is subject to tuning leakage (Section 7). Its harness code is in pull request #90 (run at commit `344ce45`, merged as `83f35ee`).

Shorter tool list, same tools. The full stack still sends every tool. The list is long because tools that share a group also share a description, and that description was repeated on every tool, with each schema indented. We state each group's shared description once and print the schemas without indentation. Nothing else changes. Like the state summary and the card, this is harness-side control delivered through the context. A shorter list might leave more room for the state the model needs next. That is an untested guess: the window is 16,384 tokens per request (65,536 shared across four slots), and the opening prompt is about 5,000 tokens on the original list.

A 10-task check scored 5 of 10 with the original list (run 80) and 8 of 10 with the shorter list (run 81). The recorded calls differed from the first full-stack run on tasks 55, 69, 26, 179, and 59. The plan's gate required 0 of those 10 to differ, so the gate failed. The plan had paired the test with the first full-stack run (77 of 200). After that smoke check, the comparison run was changed to a fresh rerun of the original list (run 82), which scores 71 of 200 (35.5%, interval 29.2–42.3%). The shorter list (run 83) scores 84 of 200 (42.0%, interval 35.4–48.9%). It solves 13 more tasks: 26 that the rerun misses, and it misses 13 that the rerun solves (58 both, 103 neither). An exact two-sided McNemar test gives $p = 0.053$, just short of the 0.05 level set in the plan (Table 1).

The opening prompt's median length falls from 4,923.5 tokens to 2,888, a 41% cut (41.3%). Mean context per request, counted separately, falls from 7,107 tokens to 4,990, about 30%. Invalid tool names stay at zero (0 of 1,276 calls with the shorter list, 0 of 1,244 with the original list). Invalid arguments are 2 of 1,276 against 9 of 1,244. Format errors are 1 of 1,801 model replies against 0 of 1,773. Mean server time per model request, prompt reading plus generation, with four slots, is 15.4 s with the shorter list and 18.8 s with the original list (medians 12.7 s and 15.9 s). Most of the drop is generation (16.2 s to 13.5 s). Prompt reading falls from 2.53 s to 1.86 s.

A second pair repeated the test in the opposite order, with the shorter list run first. The shorter list (run 84) scores 88 of 200 (44.0%, interval 37.3–50.9%), our best single run, and the original list (run 85) scores 72 of 200 (36.0%, interval 29.7–42.9%). The shorter list solves 26 tasks that the original list misses and misses 10 that it solves (62 both, 102 neither), $p = 0.011$, which clears the 0.05 level. The gain replicated. Across both pairs the shorter list solves 172 of 400 task runs and the original list 143 of 400. A McNemar test on the 52 task runs only the shorter list solved and the 23 only the original list solved gives $p = 0.001$, but it is optimistic, because the same 200 tasks appear in both pairs. The test we rely on counts each task once: the shorter list does better on 33 tasks and the original list on 17, with 150 ties (exact two-sided sign test, $p = 0.033$). The gain is modest, about 7 points, and repeat runs move individual tasks: between its two runs, the original

list changed result on 15 tasks and the shorter list on 18.

The second pair shows the same costs. Mean context per request falls from 7,121 tokens to 5,001, again about 30%, and mean server time per request from 19.4 s to 15.4 s, about 20% faster. Invalid tool names and format errors are zero for both lists. Invalid arguments are 4 of 1,300 calls with the shorter list against 5 of 1,255 (0.31% and 0.40%). A separate 50-request timing check after each run gives a mean time to the first token of 166 ms with the shorter list and 334 ms with the original list (204 ms and 473 ms in the first pair).

Table 1: Shorter tool list against the original list (same model, tasks, and other harness parts). Pair 1 is run 83 against run 82. Pair 2 is run 84 against run 85, run in the opposite order. "Shorter only" counts tasks the shorter list solves and the original list misses. p is an exact two-sided McNemar test, judged at the 0.05 level set in the plan, outside Table 2. Pooled adds the task runs of both pairs; its p is optimistic because the same 200 tasks appear twice. The task-level sign test across both pairs gives 33 tasks to 17, $p = 0.033$.

	Shorter only	Original only	Both	Neither	p
Pair 1	26	13	58	103	0.053
Pair 2	26	10	62	102	0.011
Pooled	52	23	120	205	0.001

Cost: majority vote (36.5%). Voting over five samples on top of the state summary scores 73/200, four tasks fewer than the first full-stack run, which solves 24 tasks the vote misses and misses 20 it solves ($p = 0.65$). The vote takes about 7× as long to generate as the greedy full stack, from a single sampling seed. A greedy harness matches it at a fraction of the time, which matters on a local device.

How sure are we? Intervals for the harness rows are about ±6–7 pp wide (Table 5). Among the harness comparisons in Table 2, only the full stack versus JSON text ($p = 0.023$ uncorrected) and the minimal recipe versus the native API ($p = 0.033$) fall below 0.05, and neither survives correction for multiple comparisons. The ordering among the harness rows is therefore tentative, and the +8.0 pp gain is suggestive. Two repeats of the same full stack scored 71/200 and 72/200, six and five tasks below the first run. A swing of that size is as large as most gaps between harness rows, so single-run differences of that size should not be ranked. Against JSON text, the first rerun is +5.0 pp (27 tasks only the rerun solves, 17 only JSON text, $p = 0.174$). The shorter tool list is the one change we tested twice: the first pair fell just short of the 0.05 level set in the plan ($p = 0.053$), the second cleared it ($p = 0.011$), and across both the shorter list won 33 tasks to 17 ($p = 0.033$). The gain replicated, but it is modest, about 7 points. Two contrasts are robust. The best supervised fine-tune solves only 7 tasks that JSON text misses while missing 38 that it solves ($p < 0.001$, surviving correction), and the full stack's interval (32.0–45.4%) does not overlap those of the best fine-tune (10.7–20.6%) or the forced-valid output. Preference tuning (26.5%) is *not* separable from JSON text.

6 Fine-Tuning Did Not Help

Fine-tuning is the standard way to adapt a small model to a task. Here no trained model beat the untuned model: the best supervised fine-tune scored 15.0%, significantly below JSON text, and the DPO model 26.5%, within noise of it. We treat the runs as evidence about *what* goes wrong; the version history is in Appendix B.1.

Table 2: Exact two-sided McNemar tests on per-task results (200 tasks each, same task IDs). "A only" and "B only" count tasks that one condition solves and the other misses. p is uncorrected; "Corrected" marks whether it stays below 0.05 after correcting for these eight tests (Holm's method). Full stack: JSON text + state summary + stop on repeats + status card; minimal recipe: JSON text + stop on repeats + card. Exploratory comparisons outside this family are in Table 4.

A	B	A only	B only	p	Corrected
Full stack (38.5%)	Minimal recipe (35.5%)	19	13	0.38	no
Full stack	Vote of five (36.5%)	24	20	0.65	no
Full stack	State summary (34.0%)	24	15	0.20	no
Full stack	JSON text (30.5%)	30	14	0.023	no
Minimal recipe	JSON text	26	16	0.16	no
Minimal recipe	Native API (27.0%)	37	20	0.033	no
JSON text	Native API	22	15	0.32	no
Best SFT (15.0%)	JSON text	7	38	< 0.001	yes

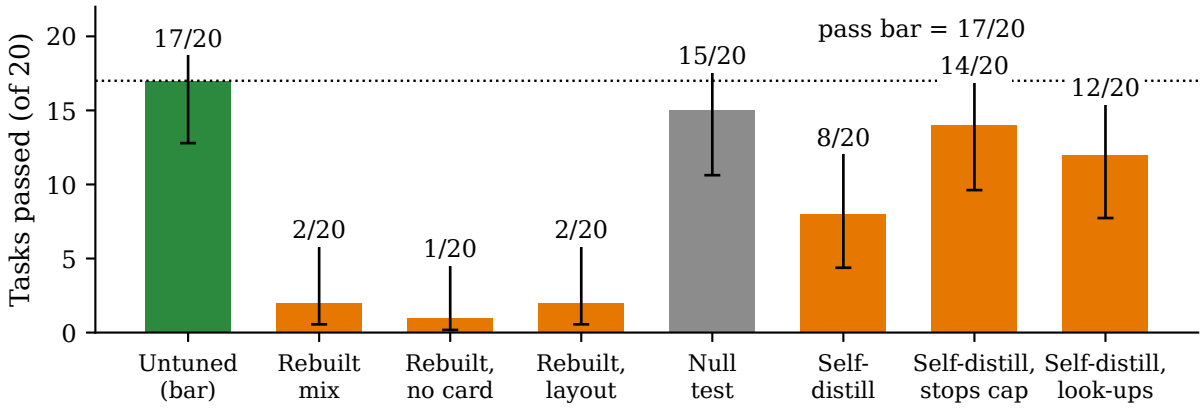


Figure 4: 20-task gate (the same 20 tasks for every bar; these are not 200-task accuracies). Error bars: Wilson 95% intervals, in tasks. Green: the untuned model with the status card, which sets the bar (17/20). Orange: fine-tuned models, run with the card except the one marked "no card". Grey: the null test. "Layout": continued training on tool lists laid out like the test; "stops cap" and "look-ups": retrains with the stop share capped or look-up episodes added. Every model trained on the rebuilt mix fails (1–2/20), while the null test stays close to the bar. Self-distillation (training on the model's own checker-approved answers to simulated tasks) and its two rebalanced retrains land in between.

Public data taught the model to stop. The first mixes scored 1/200 (0.5%). A *stop target* is a training response that is an empty list ("no further call"). After a tool result, 78.1% of targets in that data were stop targets (2,121 versus 596 continue), all from ToolACE, so the model learned to stop after its first tool result while BFCL needs further calls. Removing all of those stop targets, and adding recovery examples (an earlier empty reply, then a new request answered with a real call), scored 30/200 (15.0%), our best fine-tune: empty turns nearly vanished (167 to 2), but failures moved to state mismatch (109) and under-call (59). Adding the state summary to this fine-tune lowered it to 26/200, although the summary helps the untuned model.

A rebuilt mix collapsed. The best-validation checkpoint of the rebuilt mix scored 2/20 on the gate (Figure 4); validation loss did not predict the collapse. The collapsed model usually made one call and stopped, even when a setup step was needed (looking up a stock symbol, changing

directory, logging in), and invented missing values such as an order number. Without the card it scored 1/20, and continued training on tool lists laid out like the test still scored 2/20. To test the pipeline, we ran a *null test*, a fine-tune that should change nothing if the machinery is sound: the untuned model trained for one pass (39 updates) on 156 of its own logged answers through the same template, adapter, merge, quantization and serving steps. It scored 15/20 (interval 53–89%), within noise of the bar (17/20; 64–95%), while the rebuilt mix's 2/20 (3–30%) is far outside it. The null test rules out those pipeline steps as the cause at small update budgets. The rebuilt mix differs from it in both content and training length (22,000 steps for the evaluated checkpoint), so its data are the leading suspect but not the only one.

A candidate cause that targeted retrains did not confirm. *Self-distillation* (training on the model's own answers to simulated tasks, keeping only turns the task checker marked correct) scored 8/20; in 6 of 12 failures the model said an ID or price was missing and stopped instead of calling the tool that would supply it. Compared with the null-test data, look-up-type calls (a name-matching heuristic) fell from 37.9% to 19.7% of calls and stop targets rose from 23.1% to 33.2%. Two targeted retrains tested this: capping the stop share scored 14/20, and restoring the look-up share (39.6%) scored 12/20. Neither passed, so the data are *consistent with*, but do not establish, a shift in the decision mix (stop versus continue, look up versus act) as the cause. Leak checks found no evaluation task IDs or tool names in the null-test or self-distillation data.

7 Discussion and Limitations

State, not syntax. Across conditions, the hard part is choosing the next step from the current state of the world, including whether to stop; writing a valid call is the easy part. The levers that help support that decision: a summary of real results, a gated stop, and a card that mainly controls whether the model acts. The state summary was meant to put lost facts back in view. Empty turns fall from 41 with JSON text to 20 with the full stack, while state mismatches rise from 60 to 68. None of the levers reduces state mismatches, which remain the largest failure bucket, so the gains come from calling or stopping at the right moment, and tracking state itself remains unsolved. Levers that constrain the surface form hurt, and training appeared to shift the balance between acting and stopping rather than improve tracking.

The guided-template tie. A seven-step reasoning template on the native API also scored 61/200, tying JSON text (19 tasks swap each way; state mismatches rose to 79). The +3.5 pp channel gap may therefore reflect prompt structure as much as the channel ($p = 0.32$ in any case). Our claim is narrower: on top of JSON text, harness control adds +8.0 pp, a suggestive gain (Section 5). We did not test the harness levers on the native API or combined with the template.

What we recommend. For an on-device agent: JSON-text calls, a harness-written state summary, a gated stop rule and a status card. All four are greedy and need no training data or training compute; the minimal recipe (35.5%) is a smaller fallback. Writing the same tool list more compactly is a cheap addition: it added about 7 points across two paired runs and cut context per request by about 30% (Section 5). Before fine-tuning, screen on task-level checks rather than validation loss. Even our best single run, the full stack with the shorter list, finishes only 88 of 200, and the full stack alone moved from 77 to 71 and 72 of 200 between runs: this 2B model is not yet a reliable multi-step agent on this benchmark.

Limitations.

- **One model, one split, one device, tuned in place.** All multi-turn results use Gemma-4-E2B-it on the 200-task base split on one laptop. Settings (stop threshold, card fields, mix filters) were tuned and reported on the same tasks. Other models and the harder BFCL splits are untested.
- **Headline selection.** The full stack was assembled after seeing earlier results on the same 200 tasks (tuning leakage), and the correction family was fixed after that; its only nominally significant gain does not survive correction.
- **Sample size and decoding.** Intervals for the harness rows are about ±6–7 pp; greedy decoding with one quantization and one seed. The vote of five uses one sampling seed, and its cost was measured with four parallel slots.
- **Run-to-run movement.** The same full stack scored 77/200 on the first run and 71/200 and 72/200 on two later reruns of the original list. The shorter tool list rests on two pairs on the same 200 tasks, and between repeat runs 15 to 18 tasks changed result.
- **Heuristic error labels.** Each failed task gets one label from string matches on the checker's message, so shifts between labels suggest, but do not prove, a mechanism.
- **Gate design.** The 20 gate tasks were picked post hoc using historical per-task results from the same split, and the bar was set by the configuration it screens against, so the gate is a screening heuristic.
- **Fairness of comparisons.** The native baseline used eight server slots versus four elsewhere; the card and the vote were never run on plain JSON text. Per-task results come from a local evaluation database that is not in the repository.
- **Home-made constrained decoding.** Ours is a two-pass approximation on stock llama-server, so "forcing valid syntax hurts" may partly mean "our approximation hurts".
- **Training confounds.** Mix variants changed several factors at once; self-distillation took about 2.8× as many updates as the null test (110 versus 39), and the rebuilt mix far more.

8 Future Work and Conclusion

Future work. First, *confirmatory runs*: re-score the full stack and the JSON baseline on held-out tasks, the harder BFCL multi-turn splits and a second small model, with matched serving. Second, *retrieval-augmented tool selection*: retrieve the few relevant tools for each step instead of placing every schema in context [4, 25]. The shorter list still sends every tool; it is not this selection step. Third, *decision-balanced training data*: vary the look-up and stop shares with everything else fixed.

Conclusion. Gemma-4-E2B-it picks a single call well (772/840, 91.9%) but completes only 27.0% of multi-turn tasks through the same built-in tool-call field. Those failures are mostly empty turns (59) and wrong final states (68), with 19 under-calls, and forcing valid output makes them worse. Four greedy harness changes reached 38.5% on the first run, on par with a five-sample vote at a seventh of the time. Writing the same tool list more compactly helped modestly, and the gain replicated: about 7 points across two paired runs (84 and 88 of 200 against 71 and 72), with the shorter list winning 33 tasks to 17 ($p = 0.033$). Our best single run is 88 of 200. The gains come from calling or stopping at the right moment, and wrong final states remain the largest failure type.

The +8.0 pp over JSON text is suggestive ($p = 0.023$ uncorrected; not significant after correction) and was found on the tasks used for tuning. Fine-tuning did not reach that level. A control fine-tune that should change nothing if the machinery is sound stays close to the untuned model, so the rebuilt mix's data are the leading suspect but not the only one: that mix also differed in training length. For practitioners, these harness changes are also far cheaper than voting or training.

References

[1] Gemma Team et al. Gemma 4 technical report, 2026. URL `https://arxiv.org/abs/2607.02770`.

[2] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley Function Calling Leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 48371–48392. PMLR, 2025.

[3] ggml-org. llama.cpp: LLM inference in C/C++. `https://github.com/ggml-org/llama.cpp`, 2024.

[4] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. In *Advances in Neural Information Processing Systems*, volume 37, pages 126544–126565, 2024. doi: 10.52202/079017-4020.

[5] Yalda Taheri, Mohammad Hassan Heydari, Erfan Naaman, and Afsaneh Fatemi. Small reasoning models are instruction followers in function calling. *arXiv preprint arXiv:2608.22472*, 2026.

[6] Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*, 2023.

[7] Linzhang Li, Yixin Dong, Guanjie Wang, Ziyi Xu, Alexander Jiang, and Tianqi Chen. XGrammar-2: Dynamic and efficient structured generation engine for agentic LLMs. In *Proceedings of the ACM Conference on AI and Agentic Systems*, CAIS '26, pages 1009–1022, 2026. doi: 10.1145/3786335.3813124.

[8] Minghui Yu, Ke Mu, and Gang Wu. PTC-Decoder: Towards intelligent SLMs on offline resource-constrained edge devices. *arXiv preprint arXiv:2609.30836*, 2026.

[9] Zhi Rui Tam, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung-yi Lee, and Yun-Nung Chen. Let me speak freely? A study on the impact of format restrictions on large language model performance. In *Empirical Methods in Natural Language Processing: Industry Track*, pages 1218–1236, 2024. doi: 10.18653/v1/2024.emnlp-industry.91.

[10] Minki Kang, Wei-Ning Chen, Dongge Han, Huseyin A. Inan, Lukas Wutschitz, Yanzhi Chen, Robert Sim, and Saravan Rajmohan. ACON: Optimizing context compression for long-horizon LLM agents. In *International Conference on Machine Learning*, volume 306 of *Proceedings of Machine Learning Research*, pages 55321–55375. PMLR, 2026.

[11] Rongfeng Guo, Yinxuan Huang, Yusen Wu, Maoqing Zhong, Yunlu Chen, Meng Tang, Teng Long, and Vincent Tao Hu. From state to action: OODA-Tool for reliable multi-turn tool use. *arXiv preprint arXiv:2608.24368*, 2026.

[12] Hy Dang, Tianyi Liu, Zhuofeng Wu, Jingfeng Yang, Haoming Jiang, Tao Yang, et al. Improving large language models function calling and interpretability via guided-structured templates. In *Empirical Methods in Natural Language Processing*, pages 24426–24442, 2025. doi: 10.18653/v1/2025.emnlp-main.1242.

[13] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling parameters for reasoning. In *International Conference on Learning Representations (ICLR)*, 2025.

[14] Jianguo Zhang, Tian Lan, Ming Zhu, Zuxin Liu, Thai Hoang, Shirley Kokane, et al. xLAM: A family of large action models to empower AI agent systems. In *Nations of the Americas Chapter of the Association for Computational Linguistics*, pages 11583–11597, 2025. doi: 10.18653/v1/2025.naacl-long.578.

[15] Qiqiang Lin, Muning Wen, Qiuying Peng, Guanyu Nie, Junwei Liao, Xiaoyun Mo, Jiamu Zhou, Cheng Cheng, Yin Zhao, Jun Wang, and Weinan Zhang. Robust function-calling for on-device language model via function masking. In *International Conference on Learning Representations (ICLR)*, pages 85446–85466, 2025.

[16] Weiwen Liu, Xu Huang, Xingshan Zeng, Xinlong Hao, Shuai Yu, Dexun Li, et al. ToolACE: Winning the points of LLM function calling. In *International Conference on Learning Representations (ICLR)*, 2025.

[17] Zuxin Liu, Thai Hoang, Jianguo Zhang, Ming Zhu, Tian Lan, Shirley Kokane, et al. APIGen: Automated PIpeline for generating verifiable and diverse function-calling datasets. In *Advances in Neural Information Processing Systems*, volume 37, pages 54463–54482, 2024. doi: 10.52202/079017-1725. Datasets and Benchmarks Track.

[18] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. URL `https://openreview.net/forum?id=nZeVKeeFYf9`.

[19] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[20] Sunghee Jung, Donghun Lee, Shinbok Lee, Gaeun Seo, Daniel Lee, Byeongil Ko, Junrae Cho, Kihyun Kim, EungGyun Kim, and Myeongcheol Shin. DiaTool-DPO: Multi-turn direct preference optimization for tool-augmented large language models. In *Annual Meeting of the Special Interest Group on Discourse and Dialogue*, pages 397–416, 2025. doi: 10.18653/v1/2025.sigdial-1.32.

[21] Apple Machine Learning Research. MLX: An array framework for Apple silicon. `https://github.com/ml-explore/mlx`, 2023.

[22] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953.

[23] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157, 1947. doi: 10.1007/BF02295996.

[24] S. Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.

[25] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations (ICLR)*, pages 9695–9717, 2024.

A Intervention details

JSON text (IFFC). Following Appendix B of [5], the tool list is omitted from the API request; the system prompt asks for a JSON list of `{function_name, parameters}` objects in assistant text. Tool results return as user messages. Calls map to the same BFCL function strings as native calling.

State summary. After each successful tool execution, the harness writes a compact `[STATE]` block from real results (current directory, known files, last executed calls). The last two turns stay verbatim; older raw outputs are dropped. It is not a model-written summary. Example:

```
[STATE]
cwd:  /workspace/notes
files:  todo.txt, log.csv
last:  cd(folder='notes')->ok
```

Stop on repeats. After a tool result: (1) if the proposed call set equals the previous one, stop; (2) else if every proposed call was already executed in this user request *and* at least six non-empty tool steps have occurred in it, stop; (3) otherwise execute. The six-step window resets on a new user request. The ungated variant applies rule (2) with no minimum.

Status card (tag `[OODA]`). Prefixed to a working copy of the latest user message (the stored transcript is unchanged), stacked on stop on repeats. Fields: `goal` (latest user question, truncated); `slots` (up to eight key–value facts from executed arguments and results); `last` (last executed call and outcome); `next` (advisory: `refuse` if no tools are available, `ask` if the last result signals a missing or invalid argument, `final` if the user's message is a closing "thanks/done", otherwise `call`). The hint does not constrain decoding and the card never reads reference trajectories. It is not the multi-pass OODA-Tool of [11]. Example:

```
[OODA]
goal:  lock the doors and start the engine
slots:  lockStatus=locked
last:  startEngine()->error:  brake pedal must be pressed
next:  call
```

Full stack. The state summary, stop on repeats and status card together: the summary is written into the stored transcript after each executed call, and the card is prefixed to a working copy of the latest user message, as above.

Vote of five. On the state-summary configuration, each step samples five completions at temperature 0.7, majority-votes the `(function_name, arguments)` tuple, executes the winner once, and continues. It is not a learned verifier.

Constrained decoding. Two requests per call: the first constrained to an empty list or a list of valid tool names, the second constraining arguments with each tool's JSON schema. No native tool grammar is active. This approximates XGrammar-2's tag-triggered dispatch [7] on stock llama-server.

Tool-name automaton. A live finite automaton restricts the tool name to valid names while arguments are free, on JSON text, not stacked on stop on repeats. Execution accuracy here is not comparable to the F1 reported by Yu et al. [8].

Training. LoRA adapters [18] on Gemma-4-E2B-it with mlx-lm; best-validation checkpoint fused and quantized to Q4_K_M; evaluated with JSON text. The rebuilt mix: 30,000 training / 1,500 validation rows; 45/40/10/5% simulated / APIGen / ToolACE / xLAM; 22% empty-list targets (all after a tool result); 41.2% of rows with multi-call targets; no evaluation tool names copied into simulations. DPO: β=0.2, LoRA, scored on JSON text. How preferred and rejected responses were paired is not in the notes used for this paper.

B Failure labels, exploratory tests and training history

Table 3: Primary error per failed task (200 tasks). NFC = native API; IFFC = JSON text; Stop = stop on repeats (gated); Ungated = stop on first repeat; Summary = state summary; Minimal = minimal recipe; Full = full stack; Vote = vote of five; CD = constrained decoding; Autom. = tool-name automaton; SFT = best fine-tune. Labels as defined in Section 3.

Label	NFC	IFFC	Stop	Ungated	Summary	Minimal	Full	Vote	CD	Autom.	SFT
Empty turn	59	41	38	40	31	17	20	36	14	17	2
Under-call	19	38	37	31	37	41	35	38	70	54	59
State mismatch	68	60	60	75	64	71	68	53	95	102	109
Failed	146	139	135	146	132	129	123	127	179	173	170

Table 4: Exploratory McNemar tests, outside the eight-test correction family (uncorrected p; columns as in Table 2).

A	B	A only	B only	p
State summary (34.0%)	JSON text (30.5%)	20	13	0.30
Vote of five (36.5%)	State summary	17	12	0.46
Stop on repeats (32.5%)	JSON text	11	7	0.48
Minimal recipe (35.5%)	Stop on repeats	26	20	0.46

B.1 Training history

Version history. Dropping only mid-dialogue empty targets from the first mix still scored 1/200; deleting trailing empty answers at serving time scored 4/200. After the repaired mix (30/200), two further filtering variants scored 23/200 and 26/200 (Table 5); each changed several filters at once, so they are not single-factor ablations. The rebuilt mix's training was stopped early at step 26,500 of 32,000; the best validation loss (0.030) was at step 22,000, and that checkpoint is the one evaluated. Continued training for 2,500 steps on tool lists laid out like the test scored 2/20.

Self-distillation counts. Look-up-type calls: 67/177 in the null-test data versus 78/395 in the self-distillation data; stop targets: 36/156 versus 146/440. Capping the stop share (22.9%) left the look-up share low (20.6%). The look-up-restoring retrain lost 7 tasks the untuned model solves, and empty turns returned (9 of 66 turns). Look-up counts rest on a name-matching heuristic.

Stop targets in the rebuilt mix. Stop targets were 22% of all rebuilt-mix rows, all after a tool result, a different denominator from the first mix's 78.1% (the share of targets *after a tool result* that were stops). The rebuilt-mix models still under-called, so stop share alone does not explain the collapse.

10-task tripwire. Before the 20-task gate, trained models faced a disjoint 10-task check. Two fine-tuned checkpoints and the untuned model with the card all scored 3/10, passing the same three tasks, and across 11 strong historical 200-task runs the tripwire's rank correlation (Spearman) with the 200-task score was 0.118, a weak correlation on only 11 runs. We therefore used it only as a regression tripwire, never to rank runs.

C Full scoreboard, reproducibility and glossary

Reproducibility. Hardware: MacBook Pro, Apple M4 Pro, 48 GB. Earlier runs in this paper are recorded as llama.cpp build 8111 (commit `11c325c6e`). Runs 78 and 80–85 used commit `d344123fe`, with `llama-server -jinja -fa on -ngl 99 -c 65536 -temp 0.0 -np 4 -cb -ub 1024`. The native-API baseline used `-np 8`. BFCL v4 via the `bfcl-eval` package (exact version per run not logged). Temperature 0 except voting (0.7, N=5, one seed). The full stack (ID 78 in Table 5) is `benchmark.db scores.id` 51 with log `sft/v13/paper_evals/runs/b_fullstack_mtb200.log`; its harness code is pull request #90, run at commit `344ce45` and merged as `83f35ee`. The shorter-list runs are 80 (scores.id 59, 10-task original list, log `R81_slim/smoke_off.log`), 81 (scores.id 60, 10-task shorter list, `R81_slim/smoke_on.log`), 82 (scores.id 61, 200-task original list, `R81_slim/full_off.log`), and 83 (scores.id 62, 200-task shorter list, `R81_slim/full_on.log`). The second pair is runs 84 (scores.id 63, 200-task shorter list, `R82_confirm/full_on.log`) and 85 (scores.id 64, 200-task original list, `R82_confirm/full_off.log`). Opening-prompt lengths are medians from `R81_slim/*.metrics.jsonl`. Failure labels for the other rows are derived from the checker messages in the `eval_errors` table. Figures are regenerated from these counts by `paper/figures/src/make_refocus_figs.py`. Single-call ID 79 is the simple, multiple, and irrelevance rows in the log `sft/v13/paper_evals/runs/a_single_turn_gemma.log`; the 91.9% interval is that script's Wilson formula on 772/840.

Table 5: Every scored condition (Gemma-4-E2B-it Q4_K_M). The ID is the internal log id, with the internal name in parentheses. Interval: Wilson 95%. Rows of 20 are screening checks, not 200-task accuracies.

ID	Condition	Result	Acc.	95% interval
Multi-turn base, 200 tasks				
84	Shorter tool list on the full stack, second pair (best single run)[a]	88/200	44.0%	37.3–50.9
83	Shorter tool list on the full stack, first pair[a]	84/200	42.0%	35.4–48.9
78	Full stack, first run (`iffc-state-ooda`, PR #90)	77/200	38.5%	32.0–45.4
11	Vote of five on state summary (BoN N=5 on IFFC+STATE)	73/200	36.5%	30.1–43.4
85	Full stack, second rerun, original tool list[a]	72/200	36.0%	29.7–42.9
82	Full stack, fresh rerun, original tool list[a]	71/200	35.5%	29.2–42.3
42	Minimal recipe: card on stop on repeats ("R42")	71/200	35.5%	29.2–42.3
10	State summary (IFFC+STATE)	68/200	34.0%	27.8–40.8
30	Stop on repeats, gated at six steps (MIN=6)	65/200	32.5%	26.4–39.3
5	JSON text (IFFC)	61/200	30.5%	24.5–37.2
9	Guided template on native API (GST+NFC)	61/200	30.5%	24.5–37.2
4	Native API (NFC)	54/200	27.0%	21.3–33.5
27	Stop on first repeat (ungated)	54/200	27.0%	21.3–33.5
51	DPO, full train	53/200	26.5%	20.9–33.0
15	Best SFT, repaired mix (SFT-v4)	30/200	15.0%	10.7–20.6
46	Tool-name automaton (PTC-A)	27/200	13.5%	9.4–18.9
18	SFT, filters on first-turn targets off[†] (SFT-v6)	26/200	13.0%	9.0–18.4
16	Best SFT + state summary (SFT-v4+STATE)	26/200	13.0%	9.0–18.4
17	SFT, extra first-turn and state filters[†] (SFT-v5)	23/200	11.5%	7.8–16.7
8	Constrained decoding, two-pass (CD)	21/200	10.5%	7.0–15.5
14	SFT first mix, trailing empty answers stripped (SFT-v2 strip)	4/200	2.0%	0.8–5.0
12, 13	SFT first mixes, two variants (SFT-v1, SFT-v2)	1/200 each	0.5%	0.1–2.8
20-task gate (same 20 tasks for every row)				
69	Untuned + card: gate bar	17/20		
70	Rebuilt mix, best-validation checkpoint (SFT-v11, step 22,000)	2/20		
71	Rebuilt mix, no card	1/20		
72	Rebuilt mix, layout continue-train (SFT-v11 fmt2500)	2/20		
73	Null test (v13 null)	15/20		
74	Self-distillation, correct turns only (v13-sd)	8/20		
75	Self-distillation, stop share capped (v13-sdb)	14/20		
76	Self-distillation + simulated look-up episodes (v13-lk)	12/20		
Single-call BFCL v4 categories, native API[‡]				
79	Three categories below (weighted)	772/840	91.9%	89.9–93.6
	Simple (one function offered)	375/400	93.75%	
	Multiple (several functions; pick the right one)	183/200	91.50%	
	Irrelevance (no call needed)	214/240	89.17%	

[†]First-turn filters drop or require training examples where the first response is empty or under-specified; state filters drop continuations inconsistent with executed tool results. [‡]The two parallel-call categories are excluded because earlier runs on this server hit a tool-grammar bug that truncated multi-call output. [a]Run 80 (scores.id 59) is the 10-task original list, 5/10. Run 81 (scores.id 60) is the 10-task shorter list, 8/10. Recorded calls differed from the first full-stack run on tasks 55, 69, 26, 179, and 59. The plan's gate required 0 of 10 to differ, so the gate failed. Run 82 is the 200-task original list and run 83 the 200-task shorter list (pair 1, $p = 0.053$). Runs 84 (shorter list, run first) and 85 (original list) are pair 2 ($p = 0.011$). Across both pairs the shorter list does better on 33 tasks and worse on 17 (sign test, $p = 0.033$).

Glossary.

AST Abstract syntax tree; the single-turn BFCL matcher.
BFCL Berkeley Function Calling Leaderboard.
BoN Best-of-N; internal name for the vote of five.
DPO Direct Preference Optimization.
Full stack JSON text + state summary + stop on repeats + status card.
GST Guided-structured template: the seven-step scaffold on the native API.
Greedy decoding Always picking the most likely next token (temperature 0). With four parallel server slots, repeat runs on our server still differed (77/200, 71/200 and 72/200 for the same full stack).
Harness The program around the model that parses calls, runs tools and records state.
IFFC Instruction-followed function calling: calls written as JSON in ordinary text; called *JSON text* in this paper.
LoRA Low-rank adapters: small trainable matrices added to frozen weights.
McNemar test A paired test using only tasks where two conditions disagree.
Minimal recipe JSON text + stop on repeats + status card.
NFC Native function calling: structured `tool_calls` via the server's tool API; called *native API* in this paper.
Null test A training run that should change nothing if the pipeline is sound.
OODA Observe–orient–decide–act; here only the tag printed on the status card.
pp Percentage points.
Q4_K_M A 4-bit weight-quantization format of llama.cpp.
Self-distillation Training the model on its own answers to simulated tasks that a checker marked correct.
SFT Supervised fine-tuning.
Sign test A paired test counting the tasks on which one con-

dition solved more of its runs than the other; tasks with equal counts are dropped.

Slot One request a server processes concurrently with others.

State summary Harness-written snapshot of facts from executed results.

Status card Four-line card prefixed to the latest user message.

Stop on repeats Ends a turn when the model re-proposes executed calls.

Stop target A training response that is an empty list: no further call.

Tool-name automaton Program that constrains only the tool name.

Turn One user request and everything the model does before handing control back.

Vote of five Majority vote over five sampled calls per step.

Wilson interval A confidence interval for a proportion that stays accurate for small counts.

10-task tripwire Fixed 10-task regression check; does not rank models.

20-task gate Fixed 20-task screening check (bar 17/20); not a 200-task accuracy.